Phase 8 — Tests, sécurité, reprise, performance (SIGPN-T v2.0)

Date des essais : 8 novembre 2026. Branche `feature/sigpn-modernisation`. Données fictives uniquement.

Ce qui a été exécuté

Domaine	Moyen	Résultat
Typage	`tsc --noEmit`	sans erreur
Unitaire	`npm test`	**130** / **130**
Intégration + sécurité (serveur et base réels)	`tsx --test --test-concurrency=1 tests/integration/*.test.ts tests/security/*.test.ts`	**215** / **215** (dont 24 tests nouveaux de cette phase)
Build de production	`NEXT_DIST_DIR=.next-prod next build`	réussi ; les suites de sécurité ont aussi été rejouées contre ce build (`next start`, base de charge)
Rejeu des migrations	`prisma migrate deploy` sur une base vide	toutes appliquées ; schéma comparé à la base de développement (1 560 colonnes, 445 index, 1 458 contraintes, 45 déclencheurs, 112 énumérations) : **identique** (seule différence : des espaces dans le corps d'une fonction d'accentuation plus ancienne, sans effet)
Performance	`scripts/p8-bench.ts` (300 unités, 20 avis, 6 000 livraisons)	budgets respectés (ci-dessous)
RTL, mobile, accessibilité	audit scripté dans le navigateur à 375 px, FR et AR	voir ci-dessous

Nouveaux fichiers de tests : `tests/security/route-matrix.test.ts` (8), `tests/security/public-attacks.test.ts` (11), `tests/integration/alerts-resilience.test.ts` (5).

1. Droits (matrice générée depuis le code)

`route-matrix.test.ts` parcourt **toutes** les routes `src/app/api/**/route.ts`, en déduit chaque méthode et sa permission déclarée, puis vérifie :

- sans identifiant : 401 sur toute route non publique (une route ajoutée plus tard sans protection fait échouer le test) ;
- clé invalide, falsifiée ou vide : 401 ;
- avec un utilisateur **sans** la permission : 403 sur chaque route protégée (plus de 80 contrôles) ;
- avec un utilisateur qui **a** la permission : jamais 401/403/5xx sur les lectures ;
- aucun gestionnaire hors `withApi` (sauf liste fermée : connexion, déconnexion, santé) ;
- les permissions sensibles (`wanted:publish`, `wanted:verify`, `alerts:supervise`, `alerts:simulate`, `biometric:control`, `intake:manage`, `public:content`) ne sont pas attribuées à une majorité d'utilisateurs ;
- routes publiques : aucune ne pose de cookie, aucune n'expose de champ interne (`passwordHash`, `codeHash`, `ipHash`, identifiants d'émetteur, photo, téléphone…).

Les routes limitées en écriture déclenchent le plafond de 60 requêtes/minute par adresse : le test attend la fin de la fenêtre au lieu d'affaiblir le plafond.

2. Sécurité

Corrigé pendant la phase :

- **Octet NUL (`\u0000`) dans un paramètre ou un corps JSON : erreur 500** (PostgreSQL refuse ce caractère). Refusé désormais en 422 à l'entrée (`parseJson`, `parseQuery`) ; filet de sécurité dans `mapError` pour les autres chemins. Constaté sur `/api/v1/public/stations?q=%00`.

Vérifié (`public-attacks.test.ts`, en développement **et** contre le build de production) :

- injections SQL classiques dans la recherche publique : jamais 5xx, aucune fuite de message d'erreur ni de trace ;
- suivi : références/codes falsifiés, objets au lieu de chaînes, corps de 10 000 caractères, JSON invalide : 404/422/429 uniformes ;
- affectation de masse : un citoyen ne peut pas imposer statut, réponse publique, lien de plainte, station ou empreinte ;
- balises HTML dans une plainte : stockées comme texte inerte, réponse JSON avec `nosniff` ;
- corps mal formés, types de contenu inattendus, corps de 2 Mo : 4xx ;

TECHNOTCHAD

- identifiants hostiles dans les pages publiques (`..%2F`, NUL, 2 000 caractères) : jamais 5xx ; aucune trace dans la page 404 du build de production ;
- en-têtes présents partout (pages publiques, API, pages d'erreur) : `X-Content-Type-Options`, `X-Frame-Options: DENY`, CSP (`frame-ancestors 'none'`, `object-src 'none'`), pas de `X-Powered-By` ;
- pages internes sans connexion : redirection vers la connexion, sans contenu ;
- avis publics : seuls les champs de la liste blanche sortent, et le nombre d'avis publics égale celui des avis actifs expressément publiés ;
- contraintes en base : avis non ACTIF jamais public ; unité publique sans date de vérification refusée ;
- analyse statique : aucun `dangerouslySetInnerHTML`, `innerHTML =`, `document.write`, `eval(` dans `src/`.

Contrôle CSRF des sessions par cookie et verrouillages de connexion : déjà couverts par `tests/security/auth.test.ts` (inchangé, passe).

3. Connectivité et reprise du réseau d'alertes

Corrigé pendant la phase :

- **Passes de maintenance simultanées** (deux instances de l'application, ou l'application et un `cron`) : chacune lisait les mêmes livraisons en retard, les escaladait et notifiait les superviseurs — **trois passes simultanées produisaient 42 escalades pour 14 livraisons**. La passe s'exécute maintenant dans une transaction qui prend un verrou consultatif PostgreSQL (`pg_try_advisory_xact_lock`) ; une passe qui démarre pendant qu'une autre travaille ne fait rien (`skipped`). Le commentaire du planificateur annonçait déjà « plusieurs instances n'ont jamais de doublon » ; c'est désormais vrai.

Vérifié (`alerts-resilience.test.ts` et `alerts.test.ts`) :

- une longue panne accumule cinq avis en file durable ; à la reconnexion les cinq arrivent, une seule livraison par unité et par avis, et la reprise `afterSeq` ne renvoie rien de déjà vu ;
- 15 synchronisations et 10 battements simultanés, puis 10 accusés de réception simultanés : aucune erreur, une seule ligne de livraison, un seul accusé ;
- clé révoquée et compte désactivé : coupés immédiatement (401) ;
- toute livraison en attente vers une unité active a une prochaine tentative planifiée : rien ne dépend de la mémoire du serveur (redémarrage sans perte) ;
- l'avis est immuable (déclencheur), la retenue d'une unité hors ligne, le rattrapage cumulatif, le retrait diffusé et l'escalade unique sont couverts par les 11 tests existants.

Limite : la reprise après redémarrage est démontrée par construction (file en base, aucun état en mémoire) et par les tests ci-dessus, non par un arrêt brutal du processus en production.

4. Migrations, sauvegarde, restauration

- Les trois migrations de la modernisation (`national_alerts`, `biometric_control`, `public_portal`) sont **additives** : aucune ne contient `DROP TABLE`, `DROP COLUMN`, `TRUNCATE` ou `DELETE` ; la procédure de retour arrière est écrite en tête de chacune.
- Rejeu complet de toutes les migrations sur une base vide : réussi, schéma identique (voir tableau).
- **Non réalisé ici** : un cycle `pg_dump` / `pg_restore`. Ce poste n'a pas les outils client PostgreSQL (le moteur embarqué n'a que le serveur). Les scripts `deploy/backup/backup.sh` et `restore.sh` existent, mais **`package.json` référence `scripts/backup.ts` et `scripts/restore.ts` qui n'existent pas** (`npm run db:backup` échoue). À tester sur le serveur de déploiement avant la mise en service ; à corriger ou retirer les deux entrées.

5. Performance

Environnement : build de production (`next start`), base PostgreSQL 18 embarquée sur le même poste, 300 unités actives, 20 avis nationaux (6 000 livraisons), 30 s de sondage à 15 synchronisations par seconde (300 unités interrogeant toutes les 20 s, 18 comptes).

Mesure	n	erreurs	p50	p95	p99	budget p95
Approbation + publication vers 300 unités	20	0	67 ms	138 ms	138 ms	3 000 ms
Battement (synchronisation d'une unité)	386	0	28 ms	31 ms	34 ms	400 ms
Boîte de réception (50 avis)	386	0	11 ms	27 ms	28 ms	600 ms
Supervision nationale (300 unités)	15	0	16 ms	27 ms	27 ms	1 500 ms
Détail d'un avis (300 unités)	15	0	15 ms	21 ms	21 ms	1 500 ms

TECHNOTCHAD

Passe de maintenance sur 6 000 livraisons	1	—	404 ms			15 000 ms
Accueil public (20 simultanés)	120	0	191 ms	270 ms	271 ms	800 ms
API publique des unités / des avis	120 chacune	0	34 ms	44 / 41 ms		400 ms
Pages publiques annuaire / avis	120 chacune	0	170 ms	190 / 208 ms		800 ms

Réserves à connaître :

- les 15 synchronisations/s sont émises par 18 comptes, pas par 300 : le plafond de 240 requêtes/minute par compte l'aurait bloqué avec un seul compte. Chaque unité réelle a son propre compte, donc sa propre fenêtre ;
- le client, le serveur et la base partagent le même poste : les temps réseau d'un déploiement réel ne sont pas mesurés ;
- une passe de maintenance traite au plus 2 000 relances et 5 000 escalades ; le reste est traité à la passe suivante (une par minute par défaut). Le banc a atteint ces deux plafonds.

Pour rejouer : créer une base dont le nom finit par `_load`, y appliquer migrations et graine, puis `npm run bench:p8 -- 300 20 30` avec `DATABASE_URL` et `PMS_URL` correspondants (le script refuse toute autre base).

6. RTL, mobile, accessibilité

Audit scripté à 375 px de large sur 8 pages publiques et 5 pages internes (alertes, supervision, contrôle biométrique, personnes recherchées, site public) en français et en arabe : `dir` et `lang` corrects, un seul `h1`, aucun débordement horizontal, aucun champ sans libellé, aucun bouton ou lien sans nom, aucune image sans texte alternatif.

Corrigé : cibles tactiles trop petites (< 24 px, critère 2.5.8 de WCAG 2.2) — boutons de langue (17 → 33 px), liens de navigation publique (44 px de haut), cases à cocher du formulaire (13 → 20 px), case « signal sonore » de la bannière d'alertes et de la console biométrique, liens « Voir tout ».

Restant, faible gravité : les liens de référence dans les tableaux (par exemple `ALR-2026-…`, `PUB-2026-…`) et la case à cocher de sélection de certains tableaux font moins de 24 px de haut ; non modifié pour ne pas toucher la mise en page des tableaux existants.

Non réalisé : essai sur un appareil mobile physique, lecteur d'écran réel, mesure de contraste automatisée.

7. Autres modifications de cette phase

- `next.config.ts` : `distDir` configurable (`NEXT_DIST_DIR`) pour vérifier un build de production pendant que le serveur de développement tourne ; `.next-prod/` ajouté à `.gitignore`.
- `package.json` : script `bench:p8`.

TECHNOTCHAD